CS-471: Machine Learning

Comprehensive Exam Practice Question Bank

Weeks 1–7 Instructor: Dr. Hashir Kiani

All questions derived from weekly lecture notes

Exam Preparation Resource

Academic Year 2025–2026

Topic Summary

How to use this document:

- Each week has its own section with questions organised by topic.
- Question types: **Conceptual**, **Mathematical/Numerical**, **Applied**, and **Exam-Level Mixed**.
- **Attempt each question before reading the solution.**
- Difficulty is labelled: **[E]** = Easy, **[M]** = Medium, **[H]** = Hard / Exam-Level.
- All formulas and methods come *exactly* from the weekly lectures.

Contents

1 Week 1 — Introduction to Machine Learning & Linear Regression 2

1.1 Conceptual Questions . . . 2

1.2 Mathematical / Numerical Questions . . . 3

2 Week 2 — Gradient Descent, Evaluation Metrics & Binary Classification 5

2.1 Mathematical / Numerical Questions . . . 5

2.2 Evaluation Metrics . . . 6

2.3 Binary Classification — Score, Margin, Losses . . . 6

3 Week 3 — Logistic Regression Deep Dive 7

3.1 Loss Function and Optimisation . . . 7

4 Week 4 — Multiclass Classification & Model Evaluation 10

4.1 Multiclass Classification . . . 10

4.2 Performance Evaluation — Classification . . . 11

4.3 Overfitting and Underfitting . . . 11

5 Week 5 — Regularisation, Cross-Validation & Decision Trees 12

5.1 Cross-Validation . . . 12

5.2 Finding the Optimal Threshold for Continuous Features . . . 14

5.3 Preventing Overfitting 16

6 Week 6 — Gini, Entropy, ID3, C4.5, and CART 16

6.1 Gini Impurity 16

6.2 Entropy and Information Gain 18

6.3 Threshold Finding for Continuous Attributes — CART / Gini 18

6.4 ID3, C4.5 and CART 20

7 Week 7 — Pruning, Ensembles, Random Forests & XGBoost 21

7.1 Pruning 21

7.2 Ensemble Methods 23

8 Mixed Exam-Level Questions (All Weeks) 24

1 Week 1 — Introduction to Machine Learning & Linear Regression

1.1 Conceptual Questions

Q 1.1 [E] — Definition of Machine Learning

Define Machine Learning in your own words. What are the three core ideas that every ML system relies on?

Solution

Definition: Machine Learning is a field of study that gives computers the ability to *learn from data* without being explicitly programmed for every task.

Three core ideas:

1. **Data** — the raw material; pairs of inputs (and sometimes outputs) used for learning.
2. **Model** — a mathematical function whose *parameters* are adjusted to fit the data.
3. **Prediction** — once trained, the model generalises to new, unseen inputs.

Q 1.2 [E] — Taxonomy of ML

Describe the three main branches of Machine Learning. For each branch, give one real-world example and explain why it belongs to that branch.

Solution

1. **Supervised Learning:** Learns from labelled data (x_i, y_i).
   Example: Predicting house prices given size and location (regression).
   Why: Each training example has a known output (price); the model learns the input-output mapping.
2. **Unsupervised Learning:** Finds hidden structure in *unlabelled* data x_i.
   Example: Customer segmentation (clustering customers by purchasing behaviour without pre-defined groups).
   Why: No labels — the model must discover patterns on its own.
3. **Reinforcement Learning:** An agent takes actions and learns from rewards/penalties.
   Example: Training AlphaGo to play chess (receive $+1$ for winning, -1 for losing).
   Why: No labelled dataset; learning comes from interaction with an environment.

Q 1.3 [E] — Regression vs. Classification

Distinguish between regression and classification. For each, give the range of possible outputs and one example task.

Solution

Type	Output	Example
Regression	Continuous (any real number)	Predicting tomorrow's temperature (−10Â°C to
Classification	Discrete class labels	Spam detection ("spam" / "not spam")

Key test: **"Is the output a number from a continuous scale, or one of a fixed set of categories?"** If a number ⇒ regression; if a category ⇒ classification.

1.2 Mathematical / Numerical Questions

Q 1.4 [M] — Linear Regression Hypothesis

Write the hypothesis function for linear regression with two features x_1 and x_2. Define every symbol. If the learned weights are $w_0 = 1$, $w_1 = 2$, $w_2 = -3$, what is the prediction for a new point $\mathbf{x} = [1, 4, 2]$ (including the bias term $x_0 = 1$)?

Solution

Hypothesis function:

$$h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} = w_0 x_0 + w_1 x_1 + w_2 x_2$$

where:

- $x_0 = 1$ is always (the bias input).
- w_0 is the bias weight (intercept).
- w_1, w_2 are feature weights.
- $\mathbf{w} = [w_0, w_1, w_2]^\top$ is the weight vector.

Prediction:

$$h(\mathbf{x}) = (1)(1) + (2)(4) + (-3)(2) = 1 + 8 - 6 = \mathbf{3}$$

Q 1.5 [M] — Mean Squared Error

Given a dataset with three examples:

i	True $y^{(i)}$	Predicted $\hat{y}^{(i)}$
1	5	4
2	3	5
3	7	6

Compute the MSE and RMSE.

Solution

Step 1 — Residuals:

$$e_1 = 5 - 4 = 1, \quad e_2 = 3 - 5 = -2, \quad e_3 = 7 - 6 = 1$$

Step 2 — MSE:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2 = \frac{1^2 + (-2)^2 + 1^2}{3} = \frac{1+4+1}{3} = \frac{6}{3} = \mathbf{2.0}$$

Step 3 — RMSE:

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{2.0} \approx \mathbf{1.414}$$

Interpretation: On average, predictions are about 1.41 units away from the true values.

Q 1.6 [M] — Gradient Descent Update Rule

Write the gradient descent weight update rule for linear regression using MSE loss. Starting from $\mathbf{w} = [1,\ 0]^\top$, with a single training example $\mathbf{x}^{(1)} = [1,\ 2]^\top$, $y^{(1)} = 5$, and learning rate $\alpha = 0.1$, perform **one** gradient descent update step.

Solution

The gradient descent update rule:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \alpha\,\nabla_{\mathbf{w}}\,\text{MSE}(\mathbf{w})$$

For a single example, the gradient of MSE w.r.t. $\mathbf{w}$:

$$\nabla_{\mathbf{w}}\,\text{MSE} = 2(\hat{y} - y)\,\mathbf{x}$$

Step 1 — Current prediction:

$$\hat{y} = \mathbf{w}^\top\mathbf{x} = (1)(1) + (0)(2) = 1$$

Step 2 — Residual:

$$\hat{y} - y = 1 - 5 = -4$$

Step 3 — Gradient:

$$\nabla = 2(-4)\,[1,\ 2]^\top = [-8,\ -16]^\top$$

Step 4 — Update:

$$\mathbf{w}_{\text{new}} = \begin{bmatrix}1\\0\end{bmatrix} - 0.1\begin{bmatrix}-8\\-16\end{bmatrix} = \begin{bmatrix}1+0.8\\0+1.6\end{bmatrix} = \begin{bmatrix}1.8\\1.6\end{bmatrix}$$

After one step, $\mathbf{w} = [1.8,\ 1.6]^\top$.

Q 1.7 [H] — Learning Rate Effects

Explain what happens to gradient descent convergence when the learning rate α is (a) too large and (b) too small. Describe the decreasing learning rate schedule

$\alpha_t = \alpha_0/\sqrt{t}$ and explain why it is preferred over a constant step size.

Solution

(a) α too large: Each step overshoots the minimum. The loss oscillates or *diverges* (increases indefinitely). The algorithm never converges.
(b) α too small: Each step is tiny. Convergence is correct but *extremely slow* — thousands of iterations to reach the minimum.
Decreasing schedule $\alpha_t = \alpha_0/\sqrt{t}$:

- t = number of update steps so far.
- α_0 = initial step size (tune this one value).
- Early on (t small): steps are large — fast exploration.
- Later (t large): steps shrink — precise fine-tuning.

Why preferred: It *guarantees convergence* under mild conditions and only requires tuning one scalar (α_0) instead of finding a constant α that is "just right".

2 Week 2 — Gradient Descent, Evaluation Metrics & Binary Classification

2.1 Mathematical / Numerical Questions

Q 2.1 [M] — Full GD Worked Problem

A linear regression model has weight vector $\mathbf{w} = [w_0, w_1]$. The training set is:

$$(x^{(1)} = [1, 0],\ y = 2), \quad (x^{(2)} = [1, 0],\ y = 4), \quad (x^{(3)} = [0, 1],\ y = 1)$$

If current weights are $\mathbf{w} = [3,\ 1]$ and $\alpha = 0.5$, perform one full (batch) gradient descent update step.

Solution

Step 1 — Compute predictions:

$$\hat{y}^{(1)} = 3(1) + 1(0) = 3, \quad \hat{y}^{(2)} = 3(1) + 1(0) = 3, \quad \hat{y}^{(3)} = 3(0) + 1(1) = 1$$

Step 2 — Residuals ($\hat{y} - y$):

$$e^{(1)} = 3 - 2 = 1, \quad e^{(2)} = 3 - 4 = -1, \quad e^{(3)} = 1 - 1 = 0$$

Step 3 — Gradient (sum over all examples):

$$\nabla_{w_0} = \frac{2}{3}\sum_i e^{(i)} x_0^{(i)} = \frac{2}{3}[(1)(1) + (-1)(1) + (0)(0)] = \frac{2}{3}(0) = 0$$

$$\nabla_{w_1} = \frac{2}{3}\sum_i e^{(i)} x_1^{(i)} = \frac{2}{3}[(1)(0) + (-1)(0) + (0)(1)] = 0$$

Step 4 — Update: $\mathbf{w}_{\text{new}} = [3, 1] - 0.5[0, 0] = [3, 1]$ (no change).
Weights are *already at a critical point* for this dataset — the gradient is zero.

2.2 Evaluation Metrics

Q 2.2 [M] — MAE, MSE, RMSE, R-squared

A model predicts house prices (in \$1000s). True values: $[200, 250, 300, 350]$. Predictions: $[210, 240, 310, 330]$.
Compute: **(a)** MAE **(b)** MSE **(c)** RMSE **(d)** R^2

Solution

Residuals: $e = 210-200 = 10$; $240-250 = -10$; $310-300 = 10$; $330-350 = -20$.
(a) MAE:

$$\text{MAE} = \frac{|10| + |-10| + |10| + |-20|}{4} = \frac{10+10+10+20}{4} = \frac{50}{4} = \mathbf{12.5}$$

(b) MSE:

$$\text{MSE} = \frac{100+100+100+400}{4} = \frac{700}{4} = \mathbf{175}$$

(c) RMSE $= \sqrt{175} \approx \mathbf{13.23}$
(d) R^2**:**
$\bar{y} = (200+250+300+350)/4 = 275$.
$\text{SS}_{\text{tot}} = (200-275)^2 + (250-275)^2 + (300-275)^2 + (350-275)^2 = 5625 + 625 + 625 + 5625 = 12500$.
$\text{SS}_{\text{res}} = 100 + 100 + 100 + 400 = 700$.

$$R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}} = 1 - \frac{700}{12500} = 1 - 0.056 = \mathbf{0.944}$$

$R^2 = 0.944$ means the model explains 94.4% of the variance in house prices.

2.3 Binary Classification — Score, Margin, Losses

Q 2.3 [M] — Score, Margin, and Hinge Loss

In binary linear classification, define: **(a)** score, **(b)** margin, **(c)** zero-one loss, **(d)** hinge loss.
Given $\mathbf{w} = [2, -1]$, $x = [1, 3]$, $y = +1$. Compute the score, margin, zero-one loss, and hinge loss.

Solution

Definitions:

- **Score:** $s = \mathbf{w}^\top \mathbf{x}$ — how confidently the model assigns label $+1$.
- **Margin:** $m = y \cdot s$ — positive when prediction matches label.
- **Zero-one loss:** $\mathbf{1}[m \leq 0]$ — 1 if misclassified, 0 if correct.
- **Hinge loss:** $\max(0,\ 1-m)$ — 0 if $m \geq 1$, increases linearly below 1.

Computation:

$$s = (2)(1) + (-1)(3) = 2 - 3 = -1$$

$$m = y \cdot s = (+1)(-1) = -1$$

Zero-one loss $= \mathbf{1}[-1 \leq 0] = 1$ (misclassified).
Hinge loss $= \max(0,\ 1-(-1)) = \max(0, 2) = \mathbf{2}$.
Interpretation: The point is misclassified (score is negative but label is $+1$) and the hinge loss is 2 — a large penalty.

Q 2.4 [M] — Logistic Regression Score to Probability

The logistic (sigmoid) function is $\sigma(s) = 1/(1+e^{-s})$.
Given score $s = 2$, compute $P(y = +1 \mid x)$.
If instead $s = -3$, what is $P(y = +1 \mid x)$? Interpret both results.

Solution

Case 1: $s = 2$**:**

$$P(y = +1 \mid x) = \sigma(2) = \frac{1}{1+e^{-2}} = \frac{1}{1+0.135} = \frac{1}{1.135} \approx \mathbf{0.880}$$

The model is 88% confident the label is $+1$.
Case 2: $s = -3$**:**

$$P(y = +1 \mid x) = \sigma(-3) = \frac{1}{1+e^{3}} = \frac{1}{1+20.09} \approx \frac{1}{21.09} \approx \mathbf{0.047}$$

Only 4.7% probability of $+1$ — the model strongly predicts $y = -1$ (or class 0).
Key property: Sigmoid maps any real score to $(0, 1)$, making it interpretable as a probability.

3 Week 3 — Logistic Regression Deep Dive

3.1 Loss Function and Optimisation

Q 3.1 [M] — Binary Cross-Entropy Loss

Write the binary cross-entropy (log loss) for a single training example with true label $y \in \{0, 1\}$ and predicted probability $\hat{p} = \sigma(\mathbf{w}^\top \mathbf{x})$.
Compute the loss for: **(a)** $y = 1$, $\hat{p} = 0.9$. **(b)** $y = 0$, $\hat{p} = 0.8$.

Solution

Binary Cross-Entropy loss:

$$\mathcal{L}(y, \hat{p}) = -\big[y \log \hat{p} + (1-y)\log(1-\hat{p})\big]$$

(a) $y = 1$**,** $\hat{p} = 0.9$**:**

$$\mathcal{L} = -[1 \cdot \log(0.9) + 0 \cdot \log(0.1)] = -\log(0.9) = -(-0.1054) = \mathbf{0.105}$$

Small loss — the model is confident and correct.

(b) $y = 0$, $\hat{p} = 0.8$:

$$\mathcal{L} = -[0 \cdot \log(0.8) + 1 \cdot \log(1 - 0.8)] = -\log(0.2) = -(-1.609) = \mathbf{1.609}$$

Large loss — the model predicted $\hat{p} = 0.8$ probability of class 1, but the true label was class 0.

Q 3.2 [H] — Logistic Regression Gradient Descent Update

The gradient descent update for logistic regression is:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \alpha \sum_{i=1}^{n} \big(\sigma(\mathbf{w}^\top \mathbf{x}^{(i)}) - y^{(i)}\big)\mathbf{x}^{(i)}$$

where labels are $y \in \{0, 1\}$.
Given one example: $\mathbf{x} = [1,\ 2]^\top$, $y = 1$, $\mathbf{w} = [-1,\ 0.5]^\top$, $\alpha = 0.1$.
Perform one gradient update step.

Solution

Step 1 — Compute score:

$$s = \mathbf{w}^\top \mathbf{x} = (-1)(1) + (0.5)(2) = -1 + 1 = 0$$

Step 2 — Compute sigmoid:

$$\sigma(0) = \frac{1}{1 + e^0} = \frac{1}{2} = 0.5$$

Step 3 — Compute gradient (one example):

$$\nabla = (\sigma(s) - y)\,\mathbf{x} = (0.5 - 1)\,[1,\ 2]^\top = (-0.5)[1,\ 2]^\top = [-0.5,\ -1.0]^\top$$

Step 4 — Update weights:

$$\mathbf{w}_{\text{new}} = \begin{bmatrix} -1 \\ 0.5 \end{bmatrix} - 0.1 \begin{bmatrix} -0.5 \\ -1.0 \end{bmatrix} = \begin{bmatrix} -1 + 0.05 \\ 0.5 + 0.1 \end{bmatrix} = \begin{bmatrix} -0.95 \\ 0.6 \end{bmatrix}$$

The weights move in the direction of reducing the loss on this example (score was 0 = uncertain; true label was 1, so weights shift to make future predictions more positive).

Q 3.3 [H] — Comparing Loss Functions

Complete the table below and explain when each loss is preferred.

Loss	Formula	Used In	Differentiable?
Zero-One			
Hinge			
Cross-Entropy			

Solution

Loss	Formula	Used In	Differentiable?
Zero-One	$\mathbf{1}[y \cdot s \leq 0]$	Ideal classifier	No (step function)
Hinge	$\max(0,\ 1 - y \cdot s)$	SVM-style learning	Yes, except at $m = 1$
Cross-Entropy	$-[y \log \hat{p} + (1-y)\log(1-\hat{p})]$	Logistic Regression	Yes (smooth)

When to use:

- **Zero-one:** Optimal objective but not optimisable with gradient descent (non-differentiable).
- **Hinge:** Surrogate for zero-one; focuses on margin. Good when a safety margin matters.
- **Cross-entropy:** Dense gradients even far from boundary; naturally outputs probabilities. Standard for logistic regression and neural networks.

Q 3.4 [H] — Full Classification Example

Given three training points (x: 1D feature, bias $x_0 = 1$ prepended):

$\mathbf{x}$	y (0/1)	Initial $\hat{p}$ with $\mathbf{w} = [0, 0]$
$[1,\ 2]$	1	?
$[1,\ -1]$	0	?
$[1,\ 3]$	1	?

With $\mathbf{w} = [0,\ 0]^\top$ and $\alpha = 0.5$, perform one batch gradient descent update.

Solution

Step 1 — Predictions with $\mathbf{w} = [0, 0]$:
All scores $= 0$, so all $\hat{p} = \sigma(0) = 0.5$.
Step 2 — Errors $(\hat{p} - y)$:

$$e^{(1)} = 0.5 - 1 = -0.5, \quad e^{(2)} = 0.5 - 0 = +0.5, \quad e^{(3)} = 0.5 - 1 = -0.5$$

Step 3 — Gradient (sum of $e^{(i)}\mathbf{x}^{(i)}$):

$$\nabla_{w_0} = (-0.5)(1) + (0.5)(1) + (-0.5)(1) = -0.5 + 0.5 - 0.5 = -0.5$$

$$\nabla_{w_1} = (-0.5)(2) + (0.5)(-1) + (-0.5)(3) = -1.0 - 0.5 - 1.5 = -3.0$$

Step 4 — Update:

$$\mathbf{w}_{\text{new}} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.5 \begin{bmatrix} -0.5 \\ -3.0 \end{bmatrix} = \begin{bmatrix} 0.25 \\ 1.5 \end{bmatrix}$$

After one step, $w_1 = 1.5 > 0$, so higher x values will now receive higher scores — consistent with positive examples having $x \in \{2, 3\}$.

4 Week 4 — Multiclass Classification & Model Evaluation

4.1 Multiclass Classification

Q 4.1 [E] — OvR vs. OvO

Explain the One-vs-Rest (OvR) and One-vs-One (OvO) strategies for extending binary classifiers to K classes. How many binary classifiers does each require? Which is generally preferred for large K?

Solution

One-vs-Rest (OvR): Train K binary classifiers, one per class. Classifier k treats class k as positive and all other classes as negative. At test time, each classifier outputs a score; predict the class with the *highest score*. **Number of classifiers:** K.
One-vs-One (OvO): Train one binary classifier for *every pair* of classes. At test time, each classifier votes; predict the class with the most votes. **Number of classifiers:** $\binom{K}{2} = K(K-1)/2$.
Comparison ($K = 10$): OvR needs 10; OvO needs 45.
Preference: OvR is generally preferred for large K (far fewer classifiers). OvO can be more accurate per classifier since each uses only 2 classes of data, but scales poorly.

Q 4.2 [M] — Softmax Function

Write the softmax function for class k in a K-class problem with score vector $\mathbf{s} = [s_1, s_2, \ldots, s_K]$.
Given scores $s_1 = 2.0$, $s_2 = 1.0$, $s_3 = 0.5$, compute $P(y = k \mid \mathbf{x})$ for each class.

Solution

Softmax:

$$P(y = k \mid \mathbf{x}) = \frac{e^{s_k}}{\sum_{j=1}^{K} e^{s_j}}$$

Computation: $e^{2.0} = 7.389$, $e^{1.0} = 2.718$, $e^{0.5} = 1.649$.
Sum $= 7.389 + 2.718 + 1.649 = 11.756$.

$$P(y = 1) = \frac{7.389}{11.756} \approx \mathbf{0.629}$$

$$P(y = 2) = \frac{2.718}{11.756} \approx \mathbf{0.231}$$

$$P(y = 3) = \frac{1.649}{11.756} \approx \mathbf{0.140}$$

Check: $0.629 + 0.231 + 0.140 = 1.0$. ✓
Predict class 1 (highest probability).

4.2 Performance Evaluation — Classification

Q 4.3 [H] — Confusion Matrix to All Metrics

A binary classifier is tested on 100 examples.

$$\text{TP} = 40, \quad \text{FP} = 10, \quad \text{FN} = 15, \quad \text{TN} = 35$$

Compute: (a) Accuracy (b) Precision (c) Recall (d) F1-Score (e) False Positive Rate (FPR)

Solution

(a) Accuracy:

$$\text{Acc} = \frac{\text{TP} + \text{TN}}{N} = \frac{40 + 35}{100} = \mathbf{0.75}\ (75\%)$$

(b) Precision (of all predicted positives, how many are truly positive):

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{40}{40 + 10} = \frac{40}{50} = \mathbf{0.80}$$

(c) Recall (of all actual positives, how many did we catch):

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{40}{40 + 15} = \frac{40}{55} \approx \mathbf{0.727}$$

(d) F1-Score (harmonic mean of precision and recall):

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2(0.80)(0.727)}{0.80 + 0.727} = \frac{1.163}{1.527} \approx \mathbf{0.762}$$

(e) FPR:

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = \frac{10}{10 + 35} = \frac{10}{45} \approx \mathbf{0.222}$$

FPR is the x-axis of the ROC curve.

4.3 Overfitting and Underfitting

Q 4.4 [M] — Bias-Variance Tradeoff

Define bias and variance in the context of a machine learning model. Fill in the blank table:

Model	Bias	Variance
Very simple (e.g. constant predictor)		
Very complex (e.g. high-degree polynomial)		
Balanced / well-tuned		

Solution

Bias: Error due to overly simplistic assumptions (model cannot capture true pattern = underfitting).
Variance: Error due to excessive sensitivity to training data (model memorises noise = overfitting).

Model	Bias	Variance
Very simple (constant predictor)	**High**	**Low**
Very complex (high-degree poly)	**Low**	**High**
Balanced / well-tuned	**Low**	**Low**

Total error $\approx$ Bias^2 + Variance + Irreducible noise.
Goal: find the model complexity that minimises this total — neither too simple nor too complex.

5 Week 5 — Regularisation, Cross-Validation & Decision Trees

5.1 Cross-Validation

Q 5.1 [E] — K-Fold Cross-Validation

Describe the K-fold cross-validation procedure. If a dataset has 100 examples and $K = 5$, how many examples are in each fold? What is the model trained and tested on in each of the 5 rounds? Why is K-fold preferred over a single 80/20 train/test split?

Solution

K-fold procedure:

1. Randomly shuffle the dataset and split into K equal folds of size n/K.
2. For round $k = 1, \ldots, K$:
   (a) Use fold k as the **validation set**.
   (b) Train on the remaining $K - 1$ folds.
   (c) Record the validation error.
3. Final estimate = average of the K validation errors.

With $n = 100$, $K = 5$: Each fold has 20 examples. Training uses 80, validation uses 20, per round.
Why preferred: Every example is used for both training and validation (just in different rounds). The error estimate has lower variance than a single split (we average over 5 partitions, not just 1).

Q 5.2 [M] — L1 vs. L2 Regularisation

Write the L1 and L2 regularised loss functions for logistic regression. Explain what each does to the weights and when each is preferred. Given weights $\mathbf{w} =$

$[3, -2, 0.5, 0, -1]^\top$: compute the L1 and L2 penalties (ignoring w_0).

Solution

L2 regularised loss (Ridge):

$$\mathcal{L}_{\text{L2}} = \mathcal{L}_{\text{data}} + \lambda \sum_j w_j^2$$

L1 regularised loss (Lasso):

$$\mathcal{L}_{\text{L1}} = \mathcal{L}_{\text{data}} + \lambda \sum_j |w_j|$$

Effects:

- **L2 (Ridge):** Shrinks all weights toward zero but rarely makes them *exactly* zero. Good for reducing overfitting when all features matter.
- **L1 (Lasso):** Can shrink weights to *exactly* zero — performs automatic feature selection. Good when only a few features are relevant (sparse model).

Penalties (weights $w_1 = -2,\ w_2 = 0.5,\ w_3 = 0,\ w_4 = -1$, ignoring bias $w_0 = 3$):
L1 $= |-2| + |0.5| + |0| + |-1| = 2 + 0.5 + 0 + 1 = \mathbf{3.5}$
L2 $= (-2)^2 + (0.5)^2 + (0)^2 + (-1)^2 = 4 + 0.25 + 0 + 1 = \mathbf{5.25}$

Q 5.3 [E] — Decision Tree Introduction

What is a decision tree? What are the two fundamental parameters a decision tree learns from training data? Give a real-world example of a classification decision tree and sketch what the first level of its tree might look like.

Solution

A **decision tree** is a flowchart-like model where:

- Each **internal node** tests a feature value.
- Each **branch** corresponds to the outcome of that test.
- Each **leaf node** gives a prediction (class label for classification, numeric value for regression).

Two things a decision tree learns:

1. **Which attribute to test** at each internal node.
2. **Which threshold** to use for that test (for continuous attributes).

Real-world example: Playing golf or not based on weather. First level split: "**Outlook** = ?"

- If **Overcast** ⇒ Play Golf: YES (always).
- If **Rainy** ⇒ check next attribute.
- If **Sunny** ⇒ check next attribute.

5.2 Finding the Optimal Threshold for Continuous Features

Q 5.5 [M] — What Is a Candidate Threshold?

Decision tree algorithms (C4.5, CART) can split on continuous features using a threshold t: left branch satisfies "feature $\leq t$", right branch satisfies "feature $> t$".
(a) Given a sorted list of unique values for a feature, how do you generate candidate thresholds?
(b) How many candidate thresholds exist for n distinct values?
(c) Among all candidates, which threshold is ultimately chosen?

Solution

(a) Generating candidate thresholds: Sort all unique feature values in ascending order: $v_1 < v_2 < \ldots < v_n$. The candidate thresholds are the *midpoints between every pair of consecutive values*:

$$t_i = \frac{v_i + v_{i+1}}{2}, \quad i = 1, \ldots, n-1$$

Example: unique values $\{65, 70, 72, 75\}$ give 3 candidates: 67.5, 71, 73.5.
Note: In practice, only midpoints that separate examples of *different classes* need to be tested, but testing all $n-1$ midpoints is always correct.
(b) Number of candidates: For n distinct feature values: $\mathbf{n-1}$ candidate thresholds.
(c) Choosing the best threshold: For each candidate t:

1. Split examples into $L = \{x : x \leq t\}$ and $R = \{x : x > t\}$.
2. Compute weighted impurity: $\text{Gini}_t = \frac{|L|}{n}\text{Gini}(L) + \frac{|R|}{n}\text{Gini}(R)$ (or replace Gini with entropy for ID3/C4.5).
3. Compute the impurity gain: $\text{Gain}(t) = \text{Gini}(S) - \text{Gini}_t$.

Choose $t^* = \arg\max_t \text{Gain}(t)$, i.e. the threshold that produces the **minimum weighted impurity**.

Q 5.6 [H] — Full Threshold Search: Entropy & Information Gain

A dataset of 6 examples has one continuous feature (Temperature in °F) and a binary label (Play Golf: Yes/No):

Example	Temperature (°F)	Play?
1	65	No
2	70	No
3	72	Yes
4	75	Yes
5	80	Yes
6	85	No

(a) List all candidate thresholds.
(b) Compute the Information Gain for **every** candidate threshold.
(c) Which threshold t^* does the algorithm select? Describe the resulting split and explain *why* this threshold wins intuitively.

Solution

Step 0 — Overall entropy $H(S)$:
Dataset: 3 Yes, 3 No, $n = 6$.

$$H(S) = -\frac{3}{6}\log_2\frac{3}{6} - \frac{3}{6}\log_2\frac{3}{6} = -2 \times (0.5)(-1) = \mathbf{1.0} \text{ bit}$$

Step 1 — Candidate thresholds (midpoints):

$$t_1 = 67.5, \quad t_2 = 71, \quad t_3 = 73.5, \quad t_4 = 77.5, \quad t_5 = 82.5$$

Step 2 — IG for each threshold:

$t_1 = 67.5$ (split at 67.5):
$L(\leq 67.5)$: $\{65 \to N\} = 0\text{Y}, 1\text{N} \Rightarrow H_L = 0$.
$R(> 67.5)$: $\{70, 72, 75, 80, 85\} = 3\text{Y}, 2\text{N}$:

$$H_R = -\tfrac{3}{5}\log_2\tfrac{3}{5} - \tfrac{2}{5}\log_2\tfrac{2}{5} = 0.442 + 0.529 = 0.971$$

$$H(S|t_1) = \tfrac{1}{6}(0) + \tfrac{5}{6}(0.971) = 0.809 \quad \Rightarrow \quad \text{IG} = 1.0 - 0.809 = 0.191$$

$t_2 = 71$ (split at 71):
$L(\leq 71)$: $\{65, 70\} = 0\text{Y}, 2\text{N} \Rightarrow H_L = 0$.
$R(> 71)$: $\{72, 75, 80, 85\} = 3\text{Y}, 1\text{N}$:

$$H_R = -\tfrac{3}{4}\log_2\tfrac{3}{4} - \tfrac{1}{4}\log_2\tfrac{1}{4} = 0.311 + 0.500 = 0.811$$

$$H(S|t_2) = \tfrac{2}{6}(0) + \tfrac{4}{6}(0.811) = 0.541 \quad \Rightarrow \quad \text{IG} = 1.0 - 0.541 = \mathbf{0.459}$$

$t_3 = 73.5$ (split at 73.5):
L: $\{65, 70, 72\} = 1\text{Y}, 2\text{N}$: $H_L = -\frac{1}{3}\log_2\frac{1}{3} - \frac{2}{3}\log_2\frac{2}{3} = 0.918$.
R: $\{75, 80, 85\} = 2\text{Y}, 1\text{N}$: $H_R = 0.918$ (symmetric).

$$H(S|t_3) = \tfrac{3}{6}(0.918) + \tfrac{3}{6}(0.918) = 0.918 \quad \Rightarrow \quad \text{IG} = 1.0 - 0.918 = 0.082$$

$t_4 = 77.5$ (split at 77.5):
L: $2\text{Y}, 2\text{N} \Rightarrow H_L = 1.0$. R: $1\text{Y}, 1\text{N} \Rightarrow H_R = 1.0$.

$$H(S|t_4) = 1.0 \quad \Rightarrow \quad \text{IG} = 0.0$$

$t_5 = 82.5$ (split at 82.5):
L: $\{65, 70, 72, 75, 80\} = 3\text{Y}, 2\text{N} \Rightarrow H_L = 0.971$.
R: $\{85\} = 0\text{Y}, 1\text{N} \Rightarrow H_R = 0$.

$$H(S|t_5) = \tfrac{5}{6}(0.971) + \tfrac{1}{6}(0) = 0.809 \quad \Rightarrow \quad \text{IG} = 0.191$$

Summary table:

Threshold t	Left	Right	$H(S\|t)$	IG
67.5	{N}	{N,Y,Y,Y,N}	0.809	0.191
71	**{N,N}**	**{Y,Y,Y,N}**	**0.541**	**0.459**
73.5	{N,N,Y}	{Y,Y,N}	0.918	0.082
77.5	{N,N,Y,Y}	{Y,N}	1.000	0.000
82.5	{N,N,Y,Y,Y}	{N}	0.809	0.191

(c) Optimal threshold: $t^* = 71°\mathbf{F}$ with IG $= 0.459$.
Resulting split:

- **Temperature** $\leq 71 \Rightarrow$ predict **No** (pure leaf: both are No).
- **Temperature** $> 71 \Rightarrow$ 3 Yes, 1 No; split further.

Intuitive explanation: The threshold $t = 71$ lies exactly between the last "No" (70°F) and the first "Yes" (72°F). This is the class-boundary crossing point — the left branch becomes completely pure (only No examples). A pure branch contributes zero entropy, which is the maximum possible entropy reduction, and that is why this threshold wins.

5.3 Preventing Overfitting

Q 5.4 [H] — Early Stopping

Describe the early stopping procedure for iterative learning algorithms. What is monitored? When do you stop? What is the purpose of a validation set in this context? Why is early stopping considered a regularisation technique?

Solution

Early Stopping procedure:

1. Split the data into training and validation sets.
2. After each training epoch (pass through training data):
   (a) Compute training loss.
   (b) Compute **validation loss**.
3. **Stop when the validation loss stops decreasing** (or starts increasing), even if training loss is still decreasing.
4. Return the model weights from the *lowest validation loss point*.

Why it is regularisation: As training continues, a complex model increasingly memorises training noise — training loss keeps decreasing but validation loss rises (the model "leaves" the good generalisation region). Early stopping forces the model to stop before memorisation sets in — equivalent to restricting the effective model complexity. It acts as an automatic constraint on the number of iterations (a form of implicit regularisation).

6 Week 6 — Gini, Entropy, ID3, C4.5, and CART

6.1 Gini Impurity

Q 6.1 [E] — Gini Impurity Formula

State the Gini Impurity formula for a node with K classes. What are its minimum and maximum values (for binary classification)? What do these extremes mean in plain English?

Solution

$$\text{Gini}(S) = 1 - \sum_{k=1}^{K} p_k^2$$

where p_k is the fraction of class-k examples in set S.

Range (binary, $K = 2$):

- **Minimum** $= 0$: the set is *pure* (all one class). $p_1 = 1, p_2 = 0$: Gini $= 1 - 1 - 0 = 0$.
- **Maximum** $= 0.5$: maximally mixed ($p_1 = p_2 = 0.5$): Gini $= 1 - 0.25 - 0.25 = 0.5$.

Q 6.2 [M] — Gini Split Computation (Golf Dataset)

The golf dataset has 12 examples: 7 Yes, 5 No.

The **Windy** attribute splits the dataset as:

- Windy = False: 6 examples (4 Yes, 2 No)
- Windy = True: 6 examples (3 Yes, 3 No)

(a) Compute Gini(S) for the full dataset.

(b) Compute Gini for each branch.

(c) Compute the weighted Gini(Windy).

(d) Compute the Gini Gain for Windy.

Solution

(a) Overall Gini:

$$\text{Gini}(S) = 1 - \left(\frac{7}{12}\right)^2 - \left(\frac{5}{12}\right)^2 = 1 - \frac{49+25}{144} = 1 - \frac{74}{144} \approx \mathbf{0.486}$$

(b) Branch Ginis:

$\text{Gini}_{\text{False}} = 1 - (4/6)^2 - (2/6)^2 = 1 - 16/36 - 4/36 = 1 - 20/36 \approx \mathbf{0.444}$

$\text{Gini}_{\text{True}} = 1 - (3/6)^2 - (3/6)^2 = 1 - 0.25 - 0.25 = \mathbf{0.500}$

(c) Weighted Gini:

$$\text{Gini(Windy)} = \frac{6}{12}(0.444) + \frac{6}{12}(0.500) = 0.222 + 0.250 = \mathbf{0.472}$$

(d) Gini Gain:

$$\text{GiniGain(Windy)} = 0.486 - 0.472 = \mathbf{0.014}$$

This is smaller than the Gini Gain for Outlook (0.161), confirming that Windy is the weakest split attribute.

6.2 Entropy and Information Gain

Q 6.3 [M] — Entropy and IG for Humidity

Using the golf dataset (12 examples: 7 Yes, 5 No), the Humidity attribute splits as:

- High: 7 examples (3 Yes, 4 No)
- Normal: 5 examples (4 Yes, 1 No)

Overall $H(S) = 0.980$ bits.
Compute: **(a)** $H(\text{High})$, **(b)** $H(\text{Normal})$, **(c)** $H(S \mid \text{Humidity})$, **(d)** $\text{IG}(\text{Humidity})$.

Solution

(a) $H(\textbf{High})$ **(3 Yes, 4 No out of 7):**

$$H_{\text{High}} = -\frac{3}{7}\log_2\frac{3}{7} - \frac{4}{7}\log_2\frac{4}{7} = -(0.4286)(-1.222) - (0.5714)(-0.807) = 0.524 + 0.461 = \mathbf{0.985}$$

(b) $H(\textbf{Normal})$ **(4 Yes, 1 No out of 5):**

$$H_{\text{Normal}} = -\frac{4}{5}\log_2\frac{4}{5} - \frac{1}{5}\log_2\frac{1}{5} = -(0.8)(-0.322) - (0.2)(-2.322) = 0.257 + 0.464 = \mathbf{0.722}$$

(c) Conditional entropy:

$$H(S \mid \text{Humidity}) = \frac{7}{12}(0.985) + \frac{5}{12}(0.722) = 0.574 + 0.301 = \mathbf{0.875}$$

(d) Information Gain:

$$\text{IG}(\text{Humidity}) = 0.980 - 0.875 = \mathbf{0.105}$$

Humidity is the second-best split (after Outlook with IG = 0.305).

6.3 Threshold Finding for Continuous Attributes — CART / Gini

Q 6.4a [H] — CART Threshold Search Using Gini Impurity

CART uses **Gini Impurity** to evaluate candidate thresholds for continuous features.
The dataset below records customer **Age** and whether they **Bought** a product:

Example	Age	Bought?
1	18	No
2	22	No
3	28	Yes
4	35	Yes
5	45	Yes
6	55	No

(a) Compute the overall Gini impurity $\text{Gini}(S)$.
(b) List all 5 candidate thresholds.

(c) Compute the weighted Gini and Gini Gain for **each** candidate.
(d) Which threshold does CART select? Describe the resulting split.

Solution

(a) Overall Gini: Dataset: 3 Yes, 3 No, $n = 6$.

$$\text{Gini}(S) = 1 - \left(\frac{3}{6}\right)^2 - \left(\frac{3}{6}\right)^2 = 1 - 0.25 - 0.25 = \mathbf{0.50}$$

(b) Candidate thresholds (midpoints between consecutive ages in sorted order):

$$t_1 = 20, \quad t_2 = 25, \quad t_3 = 31.5, \quad t_4 = 40, \quad t_5 = 50$$

(c) Weighted Gini for each threshold:
Weighted Gini formula: $\text{Gini}_t = \frac{|L|}{n}\text{Gini}(L) + \frac{|R|}{n}\text{Gini}(R)$, Gini Gain $= \text{Gini}(S) - \text{Gini}_t$.

$t_1 = 20$**:**
$L(\leq 20)$: $\{18 \to N\} = 0\text{Y}, 1\text{N} \Rightarrow \text{Gini}(L) = 0$.
$R(> 20)$: $\{22, 28, 35, 45, 55\} = 3\text{Y}, 2\text{N}$: $\text{Gini}(R) = 1 - (3/5)^2 - (2/5)^2 = 1 - 0.36 - 0.16 = 0.48$.

$$\text{Gini}_{t_1} = \tfrac{1}{6}(0) + \tfrac{5}{6}(0.48) = 0.40 \quad \Rightarrow \quad \text{Gain} = 0.50 - 0.40 = 0.10$$

$t_2 = 25$**:**
$L(\leq 25)$: $\{18, 22\} = 0\text{Y}, 2\text{N} \Rightarrow \text{Gini}(L) = 0$.
$R(> 25)$: $\{28, 35, 45, 55\} = 3\text{Y}, 1\text{N}$: $\text{Gini}(R) = 1 - (3/4)^2 - (1/4)^2 = 1 - 0.5625 - 0.0625 = 0.375$.

$$\text{Gini}_{t_2} = \tfrac{2}{6}(0) + \tfrac{4}{6}(0.375) = 0.25 \quad \Rightarrow \quad \text{Gain} = 0.50 - 0.25 = \mathbf{0.25}$$

$t_3 = 31.5$**:**
L: $\{18, 22, 28\} = 1\text{Y}, 2\text{N}$: $\text{Gini}(L) = 1 - (1/3)^2 - (2/3)^2 = 1 - 1/9 - 4/9 = 4/9 \approx 0.444$.
R: $\{35, 45, 55\} = 2\text{Y}, 1\text{N}$: $\text{Gini}(R) = 4/9 \approx 0.444$.

$$\text{Gini}_{t_3} = \tfrac{3}{6}(0.444) + \tfrac{3}{6}(0.444) = 0.444 \quad \Rightarrow \quad \text{Gain} = 0.056$$

$t_4 = 40$**:**
L: $2\text{Y}, 2\text{N} \Rightarrow \text{Gini}(L) = 0.50$. R: $1\text{Y}, 1\text{N} \Rightarrow \text{Gini}(R) = 0.50$.

$$\text{Gini}_{t_4} = 0.50 \quad \Rightarrow \quad \text{Gain} = 0.0$$

$t_5 = 50$**:**
L: $\{18, 22, 28, 35, 45\} = 3\text{Y}, 2\text{N} \Rightarrow \text{Gini}(L) = 0.48$.
R: $\{55\} = 0\text{Y}, 1\text{N} \Rightarrow \text{Gini}(R) = 0$.

$$\text{Gini}_{t_5} = \tfrac{5}{6}(0.48) + \tfrac{1}{6}(0) = 0.40 \quad \Rightarrow \quad \text{Gain} = 0.10$$

Summary table:

t	Left	Right	Weighted Gini	Gini Gain
20	{N}	{N,Y,Y,Y,N}	0.400	0.100
25	**{N,N}**	**{Y,Y,Y,N}**	**0.250**	**0.250**
31.5	{N,N,Y}	{Y,Y,N}	0.444	0.056
40	{N,N,Y,Y}	{Y,N}	0.500	0.000
50	{N,N,Y,Y,Y}	{N}	0.400	0.100

(d) CART selects $t^* = 25$ (Age ≤ 25), Gini Gain $= 0.25$.

- **Left (Age ≤ 25):** Examples 1, 2 — both No $\Rightarrow$ **pure leaf, predict No**.
- **Right (Age > 25):** Examples 3, 4, 5, 6 — 3 Yes, 1 No $\Rightarrow$ impure, needs further splitting.

Why $t = 25$ wins: It is the only threshold that creates a **completely pure left branch** (only No examples age 18 and 22). A Gini of 0 on any branch is the best possible outcome and drives the largest impurity reduction. This mirrors the entropy result in Q5.6: *a threshold that creates a pure child always maximises the impurity gain.*

6.4 ID3, C4.5 and CART

Q 6.4 [M] — Gain Ratio for C4.5

The Gain Ratio used by C4.5 is: $\text{GainRatio}(S, A) = \text{IG}(S, A)/\text{SplitInfo}(S, A)$ where $\text{SplitInfo}(S, A) = -\sum_v (|S_v|/|S|) \log_2(|S_v|/|S|)$.
For the golf dataset, the Outlook attribute has 3 values: Rainy (4 examples), Overcast (3 examples), Sunny (5 examples). IG(Outlook) = 0.305 bits.
(a) Compute SplitInfo(Outlook).
(b) Compute GainRatio(Outlook).
(c) Why does C4.5 use Gain Ratio instead of raw IG?

Solution

(a) SplitInfo:

$$\text{SplitInfo(Outlook)} = -\frac{4}{12}\log_2\frac{4}{12} - \frac{3}{12}\log_2\frac{3}{12} - \frac{5}{12}\log_2\frac{5}{12}$$

$$= -(0.333)(-1.585) - (0.250)(-2.0) - (0.417)(-1.263) = 0.528 + 0.500 + 0.527 = \mathbf{1.555}$$

(b) GainRatio:

$$\text{GainRatio(Outlook)} = \frac{0.305}{1.555} \approx \mathbf{0.196}$$

(c) Why Gain Ratio: Raw IG favours attributes with *many distinct values* (e.g. a unique-ID feature with 12 values gives max IG but is useless for generalisation). SplitInfo penalises attributes that create many branches — the ratio normalises out this bias. An attribute with high IG but also high SplitInfo does not necessarily have a high Gain Ratio.

Q 6.5 [H] — CART vs. ID3 vs. C4.5 Full Comparison

Compare ID3, C4.5, and CART across six properties: (1) year / origin, (2) splitting criterion, (3) split type (binary vs. multi-way), (4) continuous attributes, (5) pruning, (6) task type.

Solution

Property	ID3	C4.5	CART
Year	1986	1993	1984
Criterion	Info Gain	Gain Ratio	Gini (classif.) / MSE (regr.)
Split type	Multi-way	Multi-way	Binary only
Continuous	No	Yes (threshold)	Yes (threshold)
Pruning	None	Post-pruning	Hyperparameters
Task	Classification	Classification	Classif. + Regression

Key points to remember:

- CART is the algorithm behind `scikit-learn`'s `DecisionTreeClassifier` and `DecisionTreeRegressor`.
- C4.5 is the most complete single-tree algorithm (handles missing values, prunes, deals with continuous attributes).
- ID3 is studied for its historical and pedagogical value.

7 Week 7 — Pruning, Ensembles, Random Forests & XGBoost

7.1 Pruning

Q 7.1 [M] — Chi-Squared Pruning Calculation

A binary split produces two subsets. The full parent node has $p = 8$ positives, $n = 8$ negatives ($N = 16$). The split results in:

- Subset 1: $p_1 = 6$, $n_1 = 2$ (8 examples)
- Subset 2: $p_2 = 2$, $n_2 = 6$ (8 examples)

(a) Compute expected counts $\hat{p}_k$ and $\hat{n}_k$ for each subset.
(b) Compute the χ^2 statistic.
(c) At $\alpha = 0.05$, DF=1, should we prune? (Critical value = 3.841.)

Solution

(a) Expected counts:
For each subset k: $\hat{p}_k = p \times (p_k + n_k)/(p + n)$, $\hat{n}_k = n \times (p_k + n_k)/(p + n)$.

Both subsets have 8 examples out of 16:

$$\hat{p}_1 = \hat{p}_2 = 8 \times \frac{8}{16} = 4.0$$

$$\hat{n}_1 = \hat{n}_2 = 8 \times \frac{8}{16} = 4.0$$

(b) χ^2 statistic:

$$\chi^2 = \sum_{k=1}^{2} \left[\frac{(p_k - \hat{p}_k)^2}{\hat{p}_k} + \frac{(n_k - \hat{n}_k)^2}{\hat{n}_k} \right]$$

$$= \frac{(6-4)^2}{4} + \frac{(2-4)^2}{4} + \frac{(2-4)^2}{4} + \frac{(6-4)^2}{4} = \frac{4}{4} + \frac{4}{4} + \frac{4}{4} + \frac{4}{4} = 1+1+1+1 = \mathbf{4.0}$$

(c) Decision: $\chi^2 = 4.0 > 3.841$ (critical value at $\alpha = 0.05$, DF=1).
Reject H_0 — the attribute *is* statistically significant. **Do NOT prune** this split.

Q 7.2 [H] — C4.5 Post-Pruning Concept

Explain the 4-step post-pruning procedure used by C4.5. In particular: **(a)** How is observed error calculated at a node?
(b) Why is a confidence interval adjustment applied?
(c) What is the condition for pruning a subtree?

Solution

Step 1: Grow the full decision tree to maximum depth using Gain Ratio.
Step 2: Compute Observed Error at each node:

$$\text{ObservedError} = \frac{E}{N}$$

where E = number of misclassified training examples, N = total examples at the node. This equals 1− (fraction belonging to the majority class).
Step 3: Compute Estimated Error with confidence interval adjustment:

$$\text{EstimatedError} = \text{ObservedError} + z_{\alpha/2}\sqrt{\frac{\text{ObservedError}(1 - \text{ObservedError})}{N}}$$

(b) Why the CI adjustment: Training error is optimistic (too low), especially for small nodes. A node with 2 examples and 1 error appears to have 50% error, but the CI makes the estimate approach 75–80%+ because 2 samples provide very little statistical confidence. The adjustment makes the estimated error more conservative and penalises small nodes.
Step 4: Prune if:

$$\text{EstimatedError(leaf)} \leq \text{EstimatedError(subtree)}$$

Replace the subtree with a leaf predicting the majority class. Repeat bottom-up until no improvement is possible.

7.2 Ensemble Methods

Q 7.3 [E] — Why Decision Trees Need Ensembles

Decision trees are said to be *high-variance* models. **(a)** What does "high variance" mean in this context? **(b)** How do ensemble methods address this problem? **(c)** How are predictions combined in classification vs. regression?

Solution

(a) High variance in decision trees: A small change in the training set (e.g. adding or removing one example) can result in a *completely different* tree structure and different predictions. The model is highly sensitive to the specific training data used.

(b) How ensembles help: Train many diverse (independent) trees; combine their predictions. Different trees make *different* errors on different examples. When independent errors are averaged or voted on, they cancel out — the ensemble error is lower than any single tree's error. Key requirement: diversity (the trees must not all make the same errors).

(c) Combining predictions:

- **Classification:** Majority voting — choose the class predicted by the most trees.
- **Regression:** Averaging — take the mean of all numerical predictions.

Q 7.4 [M] — Bagging vs. Random Forest

(a) Describe the bootstrap sampling procedure. What fraction of training examples typically appears in each bootstrap sample?

(b) How does Random Forest differ from Bagging? What formula is used for the feature subset size k?

(c) Why does feature subsampling further improve performance over bagging alone?

Solution

(a) Bootstrap sampling: Draw m examples *with replacement* from a dataset of m examples. Each draw is independent. On average, about $1-(1-1/m)^m \to 1-e^{-1} \approx 63.2\%$ of distinct examples appear at least once. The remaining $\approx 36.8\%$ (out-of-bag examples) can be used for free model validation.

(b) Random Forest vs. Bagging: Both use bootstrap samples. **Additional step in Random Forest:** at each split in each tree, instead of considering all n features, only a random subset of k features is evaluated:

$$k = \lfloor \sqrt{n} \rfloor \quad \text{(classification default)}$$

(c) Why feature subsampling helps: Even with different training samples, a very strong feature (e.g. Study Hours) tends to dominate the root split in most bagged trees — leading to highly *correlated* trees whose errors are not independent. Forcing each split to choose from only k random features *decorrelates* the trees. Uncorrelated errors cancel under averaging much more effectively than correlated ones.

Q 7.5 [H] — XGBoost Algorithm and Learning Rate

Describe the 5-step XGBoost algorithm. What is the role of the **learning rate** η? What is the final prediction formula? If η is set to 1.0 and a corrector tree has residuals perfectly learned — what is the risk?

Solution

5-step XGBoost algorithm:

1. Train the first decision tree on the training data.
2. Compute **residuals** (errors) of current ensemble predictions; misclassified examples get higher weights.
3. Train a new tree to correct the residuals / focus on high-weight examples.
4. Add the new tree to the ensemble, scaled by η.
5. Repeat steps 2–4 for B iterations.

Final prediction:

$$\hat{y} = \sum_{b=1}^{B} \eta \cdot f_b(\mathbf{x})$$

where $f_b(\mathbf{x})$ is the prediction of tree b.

Role of η (learning rate / shrinkage): Each tree's contribution is scaled down by $\eta \in (0, 1]$. Small $\eta \Rightarrow$ each step is tiny; learning is conservative and typically generalises better.

Risk with $\eta = 1.0$: With $\eta = 1$, each new tree has full weight. A tree that perfectly fits the residuals adds its full prediction to the ensemble. This causes **overfitting** — the model memorises training errors rather than learning the underlying pattern. The ensemble becomes dominated by a small number of trees, losing the smoothing benefit of many small-contribution trees.

8 Mixed Exam-Level Questions (All Weeks)

QM.1 [H] — Full ML Pipeline

A company wants to predict whether a bank loan application should be approved or denied based on 5 features: income, age, credit score, debt-to-income ratio, and loan amount.

(a) What type of ML problem is this?

(b) Name two appropriate algorithms from the course.

(c) How would you evaluate the model if only 3% of applications historically default?

(d) What is "class imbalance" and why is accuracy alone a bad metric here?

Solution

(a) This is a **supervised binary classification** problem ($y \in \{\text{Approve}, \text{Deny}\}$).

(b) Two algorithms from the course:

- Logistic Regression (binary CE loss, interpretable).

- Decision Tree / Random Forest (handles mixed features well).

(c) Metrics for imbalanced data: Because 97% are approved and only 3% default, accuracy is misleading. Use:

- **Precision** and **Recall** (especially recall on the deny/default class — missing a defaulter is costly).
- **F1-Score** (balance precision and recall).
- **AUC-ROC** (measures discrimination ability at all thresholds).

(d) Class imbalance: 97% of cases are "approve". A trivial model that *always* predicts "approve" achieves 97% accuracy but catches zero defaulters. Accuracy only counts correct predictions; it doesn't account for which class is getting right. On imbalanced data, a model can have misleadingly high accuracy while being *useless* at identifying the minority class.

QM.2 [H] — Decision Tree End-to-End on Small Dataset

A 3-class classification dataset has the following 6 training examples:

Ex.	Feature A	Feature B	Class
1	High	Yes	Cat
2	High	No	Cat
3	Low	Yes	Dog
4	Low	No	Fish
5	Med	Yes	Dog
6	Med	No	Fish

(a) Compute $H(S)$ for the full dataset (equal priors).
(b) Compute IG(Feature A) using your entropy calculation.
(c) Which feature (A or B) would ID3 split on first?

Solution

(a) $H(S)$**:** $P(\text{Cat}) = 2/6$, $P(\text{Dog}) = 2/6$, $P(\text{Fish}) = 2/6$.

$$H(S) = -3 \times \frac{2}{6}\log_2\frac{2}{6} = -3 \times (0.333)(-1.585) = 3 \times 0.528 = \mathbf{1.585}$$

(b) IG(Feature A):

- $A = \text{High}$: ex 1,2 $\to$ 2 Cat, 0 Dog, 0 Fish. $H_{\text{High}} = 0$ (pure).
- $A = \text{Low}$: ex 3,4 $\to$ 0 Cat, 1 Dog, 1 Fish. $H_{\text{Low}} = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1.0$ bit.
- $A = \text{Med}$: ex 5,6 $\to$ 0 Cat, 1 Dog, 1 Fish. $H_{\text{Med}} = 1.0$ bit.

$$H(S|A) = \frac{2}{6}(0) + \frac{2}{6}(1.0) + \frac{2}{6}(1.0) = 0 + 0.333 + 0.333 = 0.667$$

$$\text{IG}(A) = 1.585 - 0.667 = \mathbf{0.918}$$

IG(Feature B):

- $B = \text{Yes}$: ex 1,3,5 $\to$ 1 Cat, 1 Dog, 1 Fish. $H = 1.585$ bits (max entropy, no separation).
- $B = \text{No}$: ex 2,4,6 $\to$ 1 Cat, 1 Dog, 1 Fish. $H = 1.585$ bits.

$\mathrm{IG}(B) = 1.585 - 1.585 = \mathbf{0}$ bits.
(c) ID3 splits on Feature A (IG = 0.918 vs. 0). Feature B provides zero information — it perfectly mixes all three classes in both branches.

QM.3 [H] — Regression vs. Classification Tree

You are given a CART regression tree trained on study hours (input) and exam marks (output).
(a) What splitting criterion does CART use for regression?
(b) At a leaf with marks $\{40, 60, 50\}$, what is the leaf's prediction?
(c) Compute the MSE of this leaf.
(d) If splitting the leaf further yields left $\{40, 50\}$ and right $\{60\}$, compute the weighted MSE after the split. Is this split beneficial?

Solution

(a) CART regression uses **weighted Mean Squared Error (MSE)** as the splitting criterion. Choose the attribute and threshold that minimise the weighted MSE across the two child nodes.
(b) Leaf prediction: The prediction at a leaf is the **mean** of the target values (it minimises MSE):

$$\hat{y} = \frac{40 + 60 + 50}{3} = \frac{150}{3} = \mathbf{50}$$

(c) MSE of this leaf:

$$\mathrm{MSE} = \frac{(40-50)^2 + (60-50)^2 + (50-50)^2}{3} = \frac{100 + 100 + 0}{3} = \frac{200}{3} \approx \mathbf{66.67}$$

(d) Weighted MSE after split:
Left $\{40, 50\}$: $\bar{y}_L = 45$. $\mathrm{MSE}_L = [(40-45)^2 + (50-45)^2]/2 = [25 + 25]/2 = 25$.
Right $\{60\}$: $\bar{y}_R = 60$. $\mathrm{MSE}_R = 0$.

$$\text{Weighted MSE} = \frac{2}{3}(25) + \frac{1}{3}(0) = 16.67 + 0 = \mathbf{16.67}$$

Since $16.67 < 66.67$: the split **is beneficial**. MSE reduced by $66.67 - 16.67 = 50.0$.

QM.4 [H] — Random Forest vs. XGBoost Design Choices

A machine learning practitioner has two choices:
Option A: Random Forest with $B = 100$ trees, $k = \sqrt{n}$ features, bootstrap sampling.
Option B: XGBoost with $B = 100$ trees, $\eta = 0.1$, L2 regularisation.
Answer the following:
(a) Which method trains trees in parallel and which trains them sequentially? Why does this matter for training time?
(b) Which method primarily reduces variance and which primarily reduces bias?
(c) How does L2 regularisation in XGBoost differ from L2 regularisation in logistic regression?

(d) Which method would you recommend for a dataset with 1 million examples and moderate noise? Justify briefly.

Solution

(a) Parallel vs. Sequential:

- **Random Forest**: Each tree is trained *independently* on a different bootstrap sample. Training is **fully parallel** — all 100 trees can be built simultaneously on multi-core hardware.
- **XGBoost**: Each tree depends on errors from the previous one — training is **sequential**. Tree $b+1$ cannot begin until tree b has computed its residuals.

Impact: For large datasets, Random Forest is often faster in wall-clock time due to parallelism. XGBoost compensates with highly optimised implementations.

(b) Variance vs. Bias:

- **Random Forest:** Reduces **variance** by averaging independent trees (each tree has low bias, high variance; average has low variance).
- **XGBoost:** Reduces **bias** by sequentially correcting residual errors (each tree is shallow/low variance; the sequence approaches the true function, reducing bias).

(c) L2 in XGBoost vs. Logistic Regression: In logistic regression, L2 adds $\lambda\|\mathbf{w}\|^2$ to the loss — penalising large weights in the linear model. In XGBoost, L2 is applied as a penalty on the *leaf node scores* (the predicted output values in each leaf). It limits how extreme any single leaf's prediction can be, preventing any individual tree from over-correcting. The mechanism is the same (penalise large parameter values) but the "parameters" are tree leaf scores, not linear weights.

(d) Recommendation: For 1 million examples with moderate noise, **Random Forest** is recommended first:

- Parallelisable — handles large datasets faster.
- Moderate noise means variance reduction (Random Forest's strength) is the priority.
- Few hyperparameters to tune; robust out-of-the-box.
- If accuracy needs improvement after this, switch to XGBoost and tune η, depth, and regularisation.

QM.5 [H] — Full Week 5–7 Integration Problem

You are building a decision tree classifier for a 6-feature dataset with 500 training examples. The tree achieves 98% training accuracy and 62% test accuracy.

(a) What is the most likely problem? Identify it using the bias-variance framework.

(b) List **three** specific techniques from the course to fix this, explaining how each one works.

(c) If you use 5-fold cross-validation to tune the max depth hyperparameter, describe exactly how 5-fold CV works here.

(d) After converting the single tree to a Random Forest with $B = 100$ trees, training accuracy drops to 92% but test accuracy rises to 85%. Using the bias-variance framework, explain this result.

Solution

(a) Problem: Overfitting (High Variance).
The model has 98% training accuracy but only 62% test accuracy — a 36-point generalisation gap. The tree memorised training examples (including noise) but fails on unseen data. In the bias-variance framework: Low bias (fits training extremely well), High variance (sensitive to which 500 examples were drawn).

(b) Three fixes:

1. **Max depth (pre-pruning):** Limit the tree to at most d levels. Shallow trees cannot fully memorise training data. The model makes simpler, more generalisable splits.
2. **Minimum samples per leaf (`min_samples_leaf`):** Require at least k examples in every leaf. Prevents "one-example leaves" that capture noise. Each split must be based on statistically meaningful subsets.
3. **L1/L2 Regularisation or Pruning:** In CART, apply χ^2 pre-pruning or C4.5 post-pruning. Remove branches that are not statistically justified. Reduces tree size and overfitting.

(c) 5-fold CV for max depth:

1. Split 500 examples randomly into 5 folds of 100.
2. For each candidate depth $d \in \{2, 3, 4, \ldots, 15\}$:
   (a) For each fold $k = 1 \ldots 5$: train on folds $\neq k$ (400 examples), evaluate on fold k (100 examples).
   (b) Average the 5 validation accuracies.
3. Select $d^* = \arg\max_d$ avg validation accuracy.
4. Retrain the tree at depth d^* on all 500 examples.

(d) Random Forest explanation:
Training accuracy dropped because the *ensemble* no longer memorises any single training set (each tree sees a different bootstrap sample; predictions are averaged, smoothing out the memorised patterns).
Test accuracy rose dramatically because:

- 100 decorrelated trees each make different errors.
- Majority voting cancels out individual tree errors.
- Variance is drastically reduced.

The small drop in training accuracy (98% $\rightarrow$ 92%) is acceptable — it means the model is no longer *overfitting* to training details. The 23-point rise in test accuracy (62% $\rightarrow$ 85%) confirms that variance was the dominant problem, and the Random Forest successfully addresses it.